TEAM NO: 16 **PROJECT 3 PROPOSAL - MediFlow** **COURSE: S26CS6.401**

DOMAIN

Project Domain: Healthcare Domain: Streamlining Rural Healthcare Access and Coordination

DESCRIPTION OF THE USE CASE

MediFlow is an NHM(National Health Mission)-commissioned information backbone serving a cluster of 5 - 8 rural districts. It unifies fragmented data across Primary/Community Health Centres (PHCs/CHCs), district hospitals, ASHA/ANM field workers, and 108 emergency services. Rather than replacing existing infrastructure, MediFlow acts as a connective digital layer to improve care coordination, resource utilization, and outbreak surveillance.

Stakeholders & Users: Citizens, ASHA/ANM Workers, Medical Officers, 108 Dispatchers/EMTs(Emergency Medical Technicians), District/State Health Officials, and NIC/State IT Cells

Current Problems:

- Medical Officers currently lack access to ASHA field screenings, forcing consultations to "start from zero"
- 108 dispatchers send ambulances based on habit rather than real-time data regarding hospital equipment (e.g., ECG) or specialist availability
- An EMT in the 108 ambulance en route to a registered patient has no access to that patient's history like blood type, known allergies, chronic conditions. This information could be critical in the first minutes of care.
- Outbreak detection relies on weekly paper reports, often surfacing clusters 14+ days after the first presentation

Significance:

- Scalable Life-Saving Potential: NHM covers several millions of citizens. Small gains in ambulance matching and referral efficiency translate into thousands of preventable deaths averted annually
- Infrastructure Optimization: The project focuses on "connectedness" rather than procurement. It leverages existing functioning units (PHCs, 108, ASHA) by providing a shared, real-time data layer.
- Proactive Epidemic Control: With real-time symptom-stream analysis, MediFlow provides the lead time necessary to contain outbreaks like Dengue or Cholera before they spread.

KEY FUNCTIONALITIES

Primary Features & Problem Resolution

- **Interoperability Gateway:** A protocol normalization layer integrating data from IoT, labs, and field submissions into a unified schema. *Ensuring medical officers have immediate access to ASHA referral records, patient history and other data sources.*
- **108 Dispatch-Assist Integration:** A decision-support interface providing dispatchers with real-time hospital capacity, routing, and patient health data. *Replaces "habit-based" dispatching with data-driven decisions and equips EMTs with critical history during the golden hour.*
- **Bed and Resource Availability Tracking:** A facility management module for real-time updates on bed, specialist, and equipment availability. *Prevents patients from traveling hours to facilities where necessary staff or equipment (e.g., ECG machines) are unavailable.*
- **Role-Scoped Analytics Dashboards:** Permission-based dashboards providing stakeholders with operational insights using de-identified data. *Replaces delayed weekly paper reports with real-time epidemic detection and structured datasets for state-level planning.*

UI Mode

Multiple access interfaces: Android mobile app for citizens (health records, facility finder, alerts), Progressive Web App for ASHA/ANM workers (screenings, referrals, offline sync), and role-based web dashboards for PHC/CHC officers and health authorities. Additional access is provided through an ERC 108 dispatch-assist panel, SMS/IVR services for non-smartphone users, and a developer/researcher API portal.

Technical Highlights

- Frontend: React.js for the offline-first ASHA PWA. Native Android (Java) for the responsive Citizen App.
- Backend & Gateway: Java (Spring Boot) microservices handling the core business logic. Apache Camel (or Spring Integration) for robust interoperability adapters (translating MQTT, HL7, and REST).

- Event Broker: Apache Kafka (a native Java ecosystem tool) to power the high-throughput central event bus for decoupled anomaly alerts.
- Database: MongoDB to flexibly store the unified canonical event schema

Architecture & Design

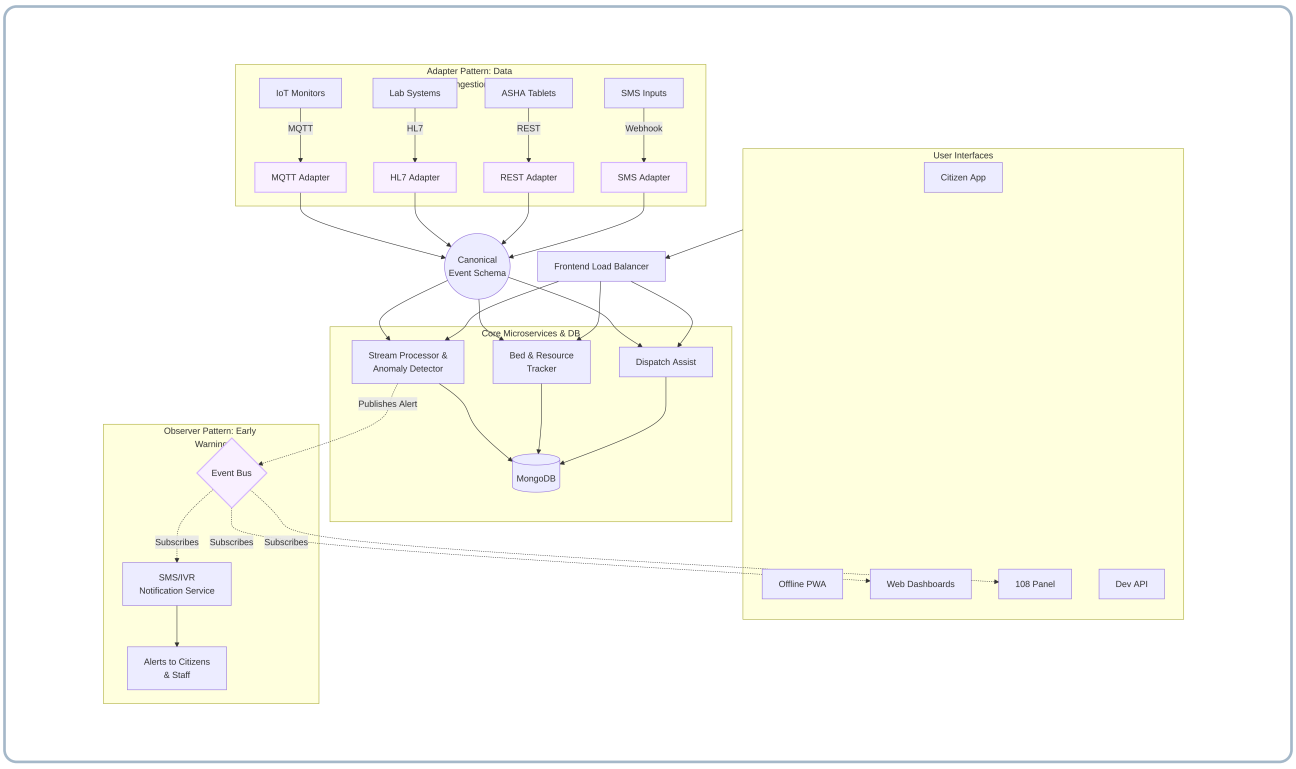
Design Patterns

- **Adapter Pattern** Interoperability gateway. Each source system - MQTT from IoT monitors, HL7/FHIR from labs, REST from ASHA tablets, SMS webhooks - gets its own adapter that translates the raw protocol into the canonical internal event schema. Downstream services see only that schema and are shielded from protocol changes in any upstream system.
- **Observer Pattern** Epidemic early-warning and real-time dashboards. When the stream processor detects a symptom-cluster anomaly, it publishes an alert event. The SMS/IVR notification service, the NHM Officer dashboard, and the 108 ERC panel all subscribe independently - none of them are coupled to each other or to the detection logic.
- **Strategy Pattern** For notifications, the citizen alert service selects a delivery strategy at runtime: push notification if the citizen has the app, SMS otherwise, IVR if SMS fails. Same interface, interchangeable implementations.

Design Decisions

- Offline-first PWA for ASHA tablets Service worker caches forms locally; IndexedDB queues submissions; Background Sync flushes on reconnect.
- SMS/IVR as first-class channels All citizen-facing alerts and reporting are available via SMS and IVR, not just the mobile app.

Architecture Diagram



EXPECTED TIME TO BUILD A PROTOTYPE (40-DAY TIMELINE)

Phase	Key Activities	Days
Research & Planning	Understanding the domain, requirements gathering, analysis and API feasibility.	6 Days
Design	Architecture design, LLD, UI/UX and DB Schema design.	14 Days
Development	Coding of 3 core non-trivial features.	8 Days
Testing	Unit and integration testing.	8 Days
Refinement	Final bug fixes and UI polishing.	4 Days
Total		**40 Days**